



**Instytut Informatyki
Wydział Nauk Ścisłych i Technicznych
Uniwersytet Rzeszowski**

Przedmiot:
Programowanie zespołowe

Dokumentacja projektu:
System do korepetycji Teachly

Wykonał:
Zespół projektowy CZERWONI

Prowadzący: mgr inż. Adam Szczur
Rzeszów 2026

Spis treści

1. Zespół projektowy	3
2. Specyfikacja projektu	3
2.1. Cel projektu	3
2.2. Zakres projektu	4
2.2.1. Zasoby	4
2.2.2. Produkty końcowe projektu	4
2.2.3. Harmonogram realizacji projektu	4
2.3. Wymagania stawiane aplikacji / systemowi	7
2.4. Panele / zakładki systemu, które będą oferowały potrzebne funkcjonalności	8
2.5.1. Raporty PDF	9
2.5.2. Inne dokumenty	9
2.6. Przepływ informacji w środowisku systemu	9
2.7. Użytkownicy aplikacji i ich uprawnienia	10
2.8. Interesariusze	10
3. Diagramy UML	11
3.1. Diagram przypadków użycia	11
3.2. Diagramy aktywności	12
3.3. Diagramy sekwencji	13
4. Baza danych	17
4.1. Diagram ERD	17
4.2. Skrypt do utworzenia struktury bazy danych	17
5. Wykorzystane technologie	20
6. Interfejsy systemu	21
6.1. Interfejs aplikacji mobilnej	21
6.2. Interfejs API	30
7. Bezpieczeństwo i testowanie	35
7.1. Bezpieczeństwo	35
7.2. Testy	35
8. Załączniki	36

1. Zespół projektowy

Nazwa zespołu: **CZERWONI**

Skład zespołu projektowego:

Imię i nazwisko	Pełniona rola	Zakres czynności / odpowiedzialności
Adrian Raczek	Lider	Scrum Master, Backend Developer, Frontend Developer, Designer UI
Maciej Pintal	Członek	Frontend Developer, Technical Writer
Krzysztof Zygmunt	Członek	Frontend Developer, Designer UI, Technical Writer
Szymon Barwa	Członek	QA Engineer, Designer UI, Technical Writer
Norbert Zdziarski	Członek	QA Engineer

2. Specyfikacja projektu

2.1. Cel projektu

Głównym celem projektu jest utworzenie w pełni funkcjonalnego systemu do cyfryzacji rynku korepetycji w Polsce. System usprawni proces wyszukiwania korepetytorów, umawiania lekcji i rozliczeń. Jego zastosowanie umożliwi efektywne zarządzanie korepetycjami przez uczniów i korepetytorów. System wpłynie pozytywnie również na jakość usług edukacyjnych i transparentność rynku korepetycji.

System do zarządzania korepetycjami jest mobilną aplikacją na platformę Android, której nadrzędnym celem jest **cyfryzacja i integracja rozproszonego rynku korepetycji w Polsce**. Projekt odpowiada na realny problem braku jednej, zaufanej platformy łączącej uczniów z korepetytorami.

Cele biznesowe projektu obejmują eliminację pośredników i nieformalnych kanałów rekrutacji korepetytorów (ogłoszenia, grupy na Facebooku), a tym samym skrócenie czasu potrzebnego na znalezienie odpowiedniego nauczyciela. Aplikacja umożliwia zarządzanie rezerwacjami, płatnościami i harmonogramem w jednym miejscu, co bezpośrednio przekłada się na zmniejszenie kosztów administracyjnych po stronie korepetytorów oraz wygodę użytkowników.

Cele jakościowe projektu dotyczą podniesienia standardów świadczenia usług edukacyjnych. System umożliwia uczniom wystawianie ocen i komentarzy po zakończonych lekcjach (w skali 1–5), co tworzy transparentny mechanizm weryfikacji jakości korepetytorów. Profil korepetytora zawiera informacje o wykształceniu, doświadczeniu i opiniach, co ułatwia uczniom podjęcie świadomej decyzji.

Cele technologiczne obejmują dostarczenie nowoczesnego, bezpiecznego i wydajnego systemu informatycznego. System oparty jest na architekturze mobilnej (Android 12+), zapewnia uwierzytelnianie JWT z 24-godzinnym tokenem, komunikację wyłącznie przez HTTPS/TLS, przechowywanie haseł w postaci bcrypt (min. 12 rund), a jego backend powinien obsługiwać co najmniej 500 jednoczesnych połączeń. Czas odpowiedzi API nie powinien przekraczać 1 sekundy dla typowych zapytań, a generowanie raportów PDF – 5 sekund.

Cele konkurencyjne polegają na stworzeniu kompleksowej platformy wyróżniającej się na tle istniejących rozwiązań dzięki integracji wyszukiwarki korepetytorów, systemu rezerwacji, płatności, oceniania oraz generowania raportów PDF w jednej aplikacji mobilnej. Interfejs zgodny z Material Design 3 gwarantuje intuicyjną obsługę bez konieczności zapoznawania się z instrukcją.

Inne cele projektu obejmują zapewnienie zgodności z przepisami RODO – system gromadzi wyłącznie dane niezbędne do świadczenia usługi, a użytkownik ma prawo do usunięcia konta i danych osobowych. Dostępność usługi na poziomie co najmniej 99,5% miesięcznie gwarantuje ciągłość działania platformy, a lokalizacja aplikacji w języku polskim z obsługą formatów PLN i dd.mm.yyyy odpowiada potrzebom rynku krajowego.

2.2. Zakres projektu

2.2.1. Zasoby

- Zespół programistów (3 osoby), 15 godzin pracy tygodniowo przez 10 tygodni w roku
- UI Designer, 20 godzin pracy
- Zespół testerów (2 osoby), 3 godzin pracy tygodniowo przez 10 tygodni
- Inżynier Dokumentacji Technicznej, 30 godzin pracy
- Darmowe narzędzia do wytworzenia systemu:
 - IntelliJ
 - PgAdmin
 - GitHub
 - Jira
 - Docker
 - Software Ideas Modeler

2.2.2. Produkty końcowe projektu

- *System do zarządzania korepetycjami*
 - Aplikacja mobilna Teachly
 - Docker dla części backendowej i bazy danych
 - Kod źródłowy aplikacji
 - Skrypty do utworzenia bazy danych
- Dokumentacja techniczna projektu
 - Diagramy UML
- Podręcznik użytkownika dla wytworzonego systemu

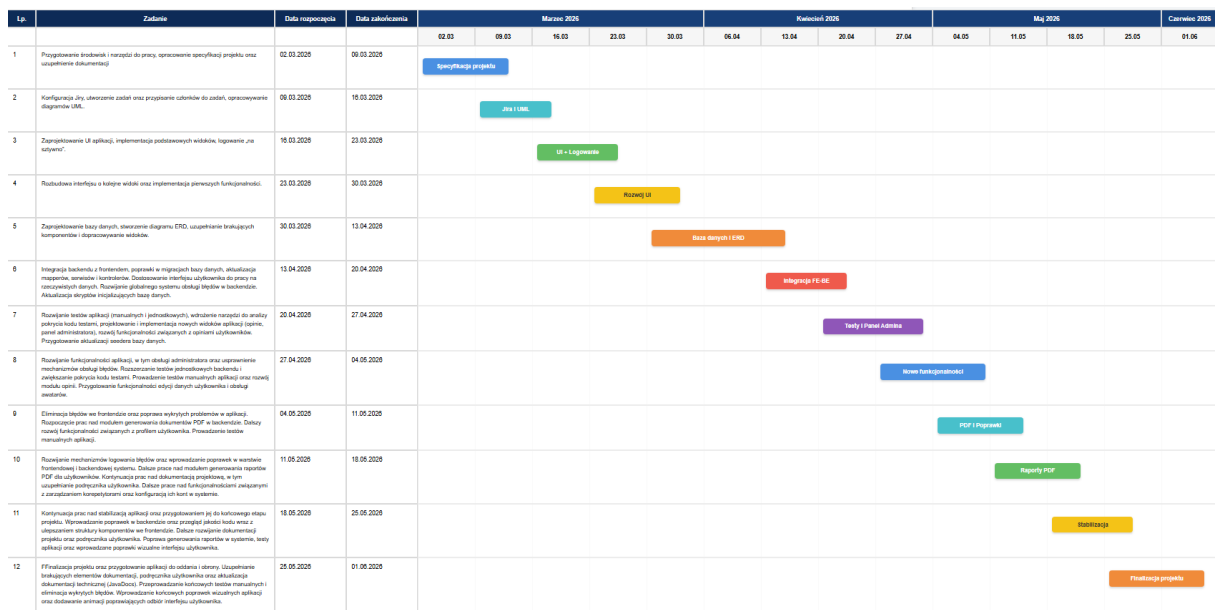
2.2.3. Harmonogram realizacji projektu

02.03.2026 - 09.03.2026	Przygotowanie środowisk i narzędzi do pracy, opracowanie specyfikacji projektu oraz uzupełnienie dokumentacji
09.03.2026 - 16.03.2026	Konfiguracja Jiry, utworzenie zadań oraz przypisanie członków do zadań, opracowywanie diagramów UML
16.03.2026 - 23.03.2026	Zaprojektowanie UI aplikacji, implementacja podstawowych widoków, logowanie "na sztywno"
23.03.2026 - 30.03.2026	Rozbudowa interfejsu o kolejne widoki oraz implementacja pierwszych funkcjonalności
30.03.2026 - 13.04.2026	Zaprojektowanie bazy danych, stworzenie diagramu ERD, uzupełnianie brakujących komponentów i dopracowywanie widoków, przygotowywanie skryptów
13.04.2026 - 20.04.2026	Integracja backendu z frontendem, poprawki w migracjach bazy danych, aktualizacja mapperów, serwisów i kontrolerów. Dostosowanie interfejsu użytkownika do pracy na rzeczywistych danych. Rozwijanie globalnego systemu obsługi błędów w backendzie. Aktualizacja skryptów inicjalizujących bazę danych.
20.04.2026 - 27.04.2026	Rozwijanie testów aplikacji (manualnych i jednostkowych), wdrożenie narzędzi do analizy pokrycia kodu testami, projektowanie i implementacja nowych widoków aplikacji (opinie, panel administratora), rozwój funkcjonalności związanych z opiniami użytkowników. Przygotowanie aktualizacji seedera bazy danych.
27.04.2026 - 04.05.2026	Rozwijanie funkcjonalności aplikacji, w tym obsługi administratora oraz usprawnienie mechanizmów obsługi błędów. Rozszerzanie

	testów jednostkowych backendu i zwiększanie pokrycia kodu testami. Prowadzenie testów manualnych aplikacji oraz rozwój modułu opinii. Przygotowanie funkcjonalności edycji danych użytkownika i obsługi awatarów.
04.05.2026 - 11.05.2026	Eliminacja błędów we frontendzie oraz poprawa wykrytych problemów w aplikacji. Rozpoczęcie prac nad modułem generowania dokumentów PDF w backendzie. Dalszy rozwój funkcjonalności związanych z profilem użytkownika. Prowadzenie testów manualnych aplikacji.
11.05.2026 - 18.05.2026	Rozwijanie mechanizmów logowania błędów oraz wprowadzanie poprawek w warstwie frontendowej i backendowej systemu. Dalsze prace nad modułem generowania raportów PDF dla użytkowników. Kontynuacja prac nad dokumentacją projektową, w tym uzupełnianie podręcznika użytkownika. Dalsze prace nad funkcjonalnościami związanymi z zarządzaniem korepetytorami oraz konfiguracją ich kont w systemie.
18.05.2026 - 25.05.2026	Kontynuacja prac nad stabilizacją aplikacji oraz przygotowaniem jej do końcowego etapu projektu. Wprowadzanie poprawek w backendzie oraz przegląd jakości kodu wraz z ulepszaniem struktury komponentów we frontendzie. Dalsze rozwijanie dokumentacji projektu oraz podręcznika użytkownika. Poprawa generowania raportów w systemie, testy aplikacji oraz wprowadzane poprawki wizualne interfejsu użytkownika.
25.05.2026 - 01.06.2026	Finalizacja projektu oraz przygotowanie aplikacji do oddania i obrony. Uzupełnianie brakujących

elementów dokumentacji, podręcznika użytkownika oraz aktualizacja dokumentacji technicznej (JavaDocs). Przeprowadzanie końcowych testów manualnych i eliminacja wykrytych błędów. Wprowadzanie końcowych poprawek wizualnych aplikacji oraz dodawanie animacji poprawiających odbiór interfejsu użytkownika.

2.2.3.1. Wykres Gantta



Wykres Gantta

2.3. Wymagania stawiane aplikacji / systemowi

Moduł uwierzytelniania (Auth)

- Rejestracja konta z wyborem roli (uczeń / korepetytor)
- Logowanie z tokenem JWT (ważność 24h, mechanizm odświeżania tokena)
- Zmiana hasła i danych profilowych
- Wylogowanie i unieważnienie tokena po stronie backendu

Moduł administracji użytkownikami

- Trzy role użytkowników: Uczeń (student), Korepetytor (tutor), Administrator (admin)
- Przeglądanie i zarządzanie wszystkimi kontami użytkowników
- Blokowanie / odblokowanie kont użytkowników
- Moderacja ocen - usuwanie nieodpowiednich treści
- Podgląd wszystkich lekcji i transakcji w systemie

Moduł raportów (PDF)

- Raport miesięczny ucznia: lista lekcji, daty, przedmioty, kwoty, suma wydatków
- Raport zarobków korepetytora: lista lekcji, stawki, suma miesięczna

- Raport systemowy dla administratora: liczba aktywnych użytkowników, lekcji, transakcji

Moduł konfiguracji i zarządzania

- Zarządzanie słownikiem przedmiotów i kategorii
- Konfiguracja poziomów nauczania (podstawowy, rozszerzony, przygotowanie do egzaminu)
- Zarządzanie formatami zajęć (online / stacjonarne)

Panel ucznia

- Wyszukiwanie korepetytorów z filtrami: przedmiot, poziom, forma, cena, ocena
- Przeglądanie profilu korepetytora: bio, wykształcenie, opinie, dostępność
- Rezerwacja lekcji - wybór terminu z kalendarza dostępności korepetytora
- Widok nadchodzących i historycznych lekcji
- Płatność za lekcję (symulacja - status pending/paid)
- Wystawienie oceny i komentarza po zakończonej lekcji
- Generowanie raportu PDF z historią lekcji i wydatkami za dany miesiąc

Panel korepetytora

- Tworzenie i edycja profilu: bio, wykształcenie, stawka godzinowa, przedmioty i poziomy
- Zarządzanie dostępnością w kalendarzu tygodniowym
- Przeglądanie i akceptowanie / odrzucanie wniosków o lekcje
- Dodawanie notatek po zakończonej lekcji (widoczne tylko dla korepetytora)
- Widok zarobków i historii płatności
- Generowanie raportu PDF z zarobkami za wybrany miesiąc

2.4. Panele / zakładki systemu, które będą oferowały potrzebne funkcjonalności

- Panel administratora
 - Główne narzędzie administratorów systemu umożliwiające wykonanie wszystkich czynności potrzebnych do zarządzania użytkownikami, moderowania treści i analizy statystyk
- Panel korepetytora
 - Tworzenie i edycja profilu: bio, wykształcenie, stawka godzinowa, przedmioty i poziomy
 - Zarządzanie dostępnością w kalendarzu tygodniowym
 - Przeglądanie i akceptowanie / odrzucanie wniosków o lekcje
 - Dodawanie notatek po zakończonej lekcji (widoczne tylko dla korepetytora)
 - Widok zarobków i historii płatności
 - Generowanie raportu PDF z zarobkami za wybrany miesiąc
- Panel użytkownika (ucznia)
 - Wyszukiwanie korepetytorów z filtrami: przedmiot, poziom, forma, cena, ocena
 - Przeglądanie profilu korepetytora: bio, wykształcenie, opinie, dostępność
 - Rezerwacja lekcji - wybór terminu z kalendarza dostępności korepetytora
 - Widok nadchodzących i odbytych lekcji
 - Płatność za lekcję (symulacja - status pending/paid)
 - Wystawienie oceny i komentarza po zakończonej lekcji
 - Generowanie raportu PDF z historią lekcji i wydatkami za dany miesiąc
- Zakładka raportów
 - Raport miesięczny ucznia: lista lekcji, daty, przedmioty, kwoty, suma wydatków

- Raport zarobków korepetytora: lista lekcji, stawki, suma miesięczna
- Raport systemowy dla administratora: liczba aktywnych użytkowników, lekcji, transakcji
- Okno ustawień
 - Zarządzanie słownikiem przedmiotów i kategorii
 - Konfiguracja poziomów nauczania (podstawowy, rozszerzony, przygotowanie do egzaminu)
 - Zarządzanie formatami zajęć (online / stacjonarne)
- Logowanie i uwierzytelnianie
 - Rejestracja konta z wyborem roli (uczeń / korepetytor)
 - Logowanie z tokenem JWT (ważność 24h, mechanizm odświeżania tokena)
 - Zmiana hasła i danych profilowych
 - Wylogowanie i unieważnienie tokena po stronie backendu

2.5. Typy wymaganych dokumentów w projekcie oraz dostęp do nich

2.5.1. Raporty PDF

2.5.1.1. Rodzaje raportów PDF

System umożliwia generowanie raportów w formacie PDF z możliwością filtrowania danych (zakres dat, użytkownik, przedmiot itp.). Każdy raport zawiera podsumowanie oraz szczegółową listę danych zgodnie z wybranymi kryteriami. Rodzaje raportów PDF:

1. Raport miesięczny ucznia

Umożliwia wygenerowanie zestawienia lekcji ucznia w wybranym okresie (np. miesiąc, zakres dat).

Możliwe filtry: *uczeń, zakres dat, przedmiot, korepetytor.*

Zawiera: listę lekcji, daty, przedmioty, liczbę godzin, koszt pojedynczej lekcji, łączną kwotę oraz sumę wydatków w danym okresie.

2. Raport zarobków korepetytora

Przedstawia zestawienie przeprowadzonych lekcji oraz uzyskanych przychodów w wybranym okresie.

Możliwe filtry: *korepetytor, zakres dat, przedmiot.*

Zawiera: listę lekcji, daty, stawki godzinowe, liczbę godzin, przychód z każdej lekcji oraz łączną sumę miesięczną.

3. Raport systemowy administratora

Raport zbiorczy przedstawiający statystyki działania systemu w wybranym okresie.

Możliwe filtry: *zakres dat.*

Zawiera: liczbę aktywnych użytkowników, liczbę uczniów i korepetytorów, liczbę przeprowadzonych lekcji, liczbę transakcji oraz łączną wartość transakcji.

2.5.2. Inne dokumenty

2.6. Przepływ informacji w środowisku systemu

System ma architekturę scentralizowaną, opartą na modelu klient–serwer. Główną składową systemu jest relacyjna baza danych PostgreSQL, przechowująca wszystkie dane aplikacji (użytkowników, lekcji, rezerwacji, płatności, ocen).

Aplikacja mobilna (klient Android) komunikuje się wyłącznie z serwerem backendowym poprzez interfejs REST API za pośrednictwem protokołu HTTPS/TLS. Backend przetwarza żądania, weryfikuje tokeny JWT i wykonuje operacje na bazie danych. Żadna część logiki biznesowej nie jest realizowana po stronie klienta.

Przepływ informacji między modułami:

- Moduł Auth przekazuje token JWT do pozostałych modułów (autoryzacja każdego żądania)
- Panel ucznia wysyła żądania do modułów: wyszukiwania, rezerwacji, płatności, ocen
- Panel korepetytora komunikuje się z modułami: profilu, kalendarza, wniosków o lekcje
- Moduł raportów pobiera dane z bazy i generuje pliki PDF po stronie serwera
- Panel administratora ma dostęp do danych wszystkich pozostałych modułów

2.7. Użytkownicy aplikacji i ich uprawnienia

- Administrator
 - Przeglądanie i zarządzanie wszystkimi kontami użytkowników
 - Blokowanie / odblokowywanie kont
 - Moderacja ocen i komentarzy (usuwanie nieodpowiednich treści)
 - Podgląd wszystkich lekcji i transakcji
 - Generowanie systemowych raportów PDF
 - Konfiguracja słowników (przedmioty, poziomy, formaty zajęć)
- Korepetytor
 - Tworzenie i edycja własnego profilu (bio, stawka, przedmioty)
 - Zarządzanie dostępnością w kalendarzu tygodniowym
 - Akceptowanie lub odrzucanie wniosków o lekcje
 - Dodawanie prywatnych notatek po lekcji
 - Przeglądanie historii swoich lekcji i zarobków
 - Generowanie raportu PDF ze swoimi zarobkami
 - Oznaczanie zakończonej lekcji jako opłaconej
- Uczeń
 - Wyszukiwanie korepetytorów z filtrami
 - Przeglądanie profili korepetytorów
 - Rezerwacja lekcji z kalendarza dostępności korepetytora
 - Wystawianie ocen i komentarzy po lekcji
 - Przeglądanie historii swoich lekcji i wydatków
 - Generowanie raportu PDF z historią swoich lekcji

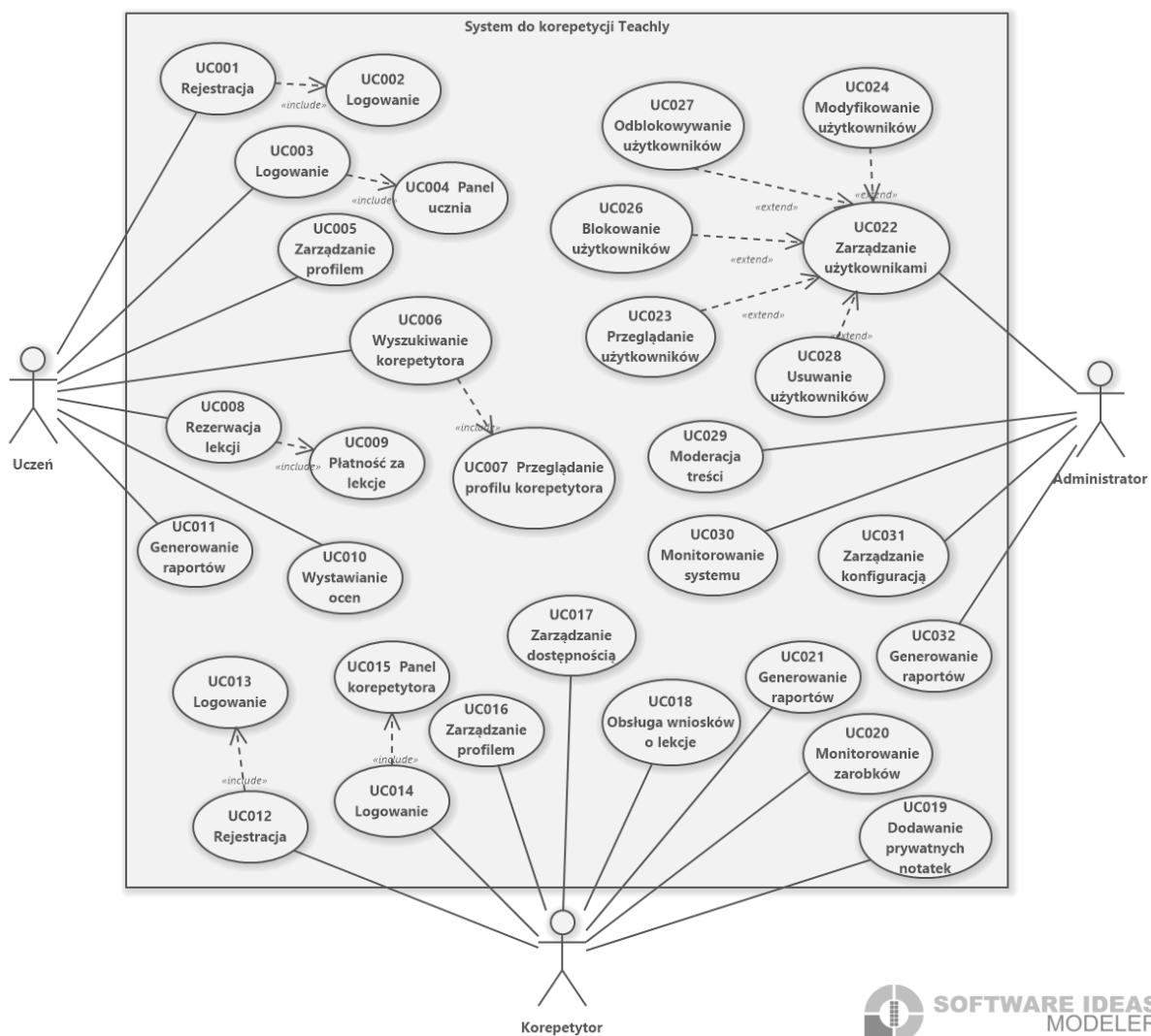
2.8. Interesariusze

- Interesariusze wewnętrzni
 - Zespół projektowy CZERWONI
 - Prowadzący przedmiot Programowanie Zespołowe
 - Instytut Informatyki UR
- Interesariusze zewnętrzni
 - Uczniowie
 - Korepetytorzy
 - Rodzice uczniów
 - Szkoły i instytucje edukacyjne

3. Diagramy UML

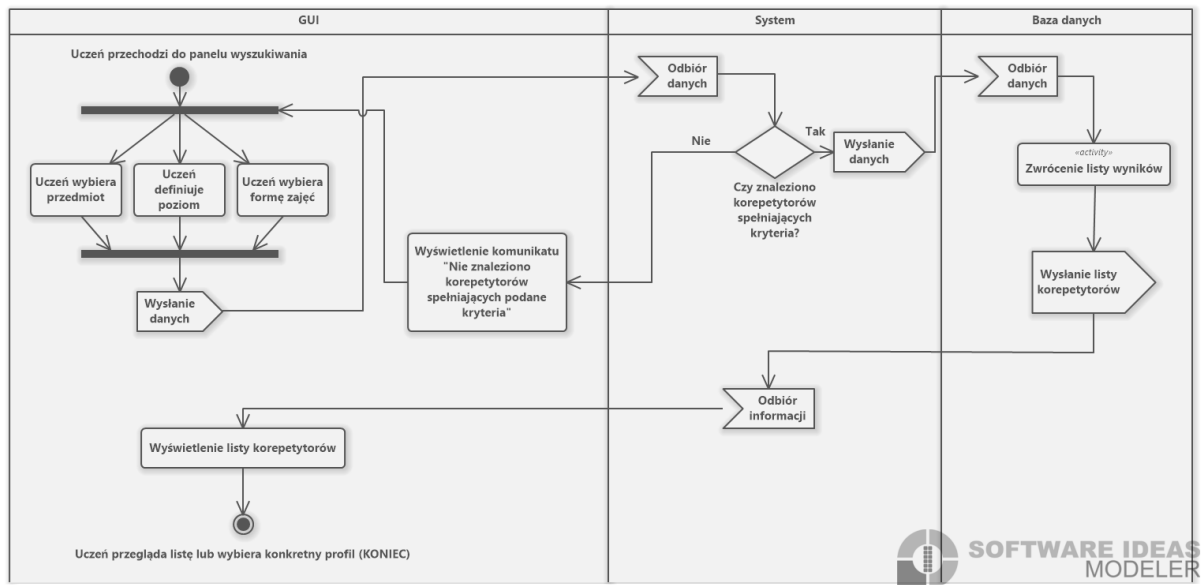
Diagramy projektowe systemu będą opracowywane w środowisku Software Ideas Modeler

3.1. Diagram przypadków użycia

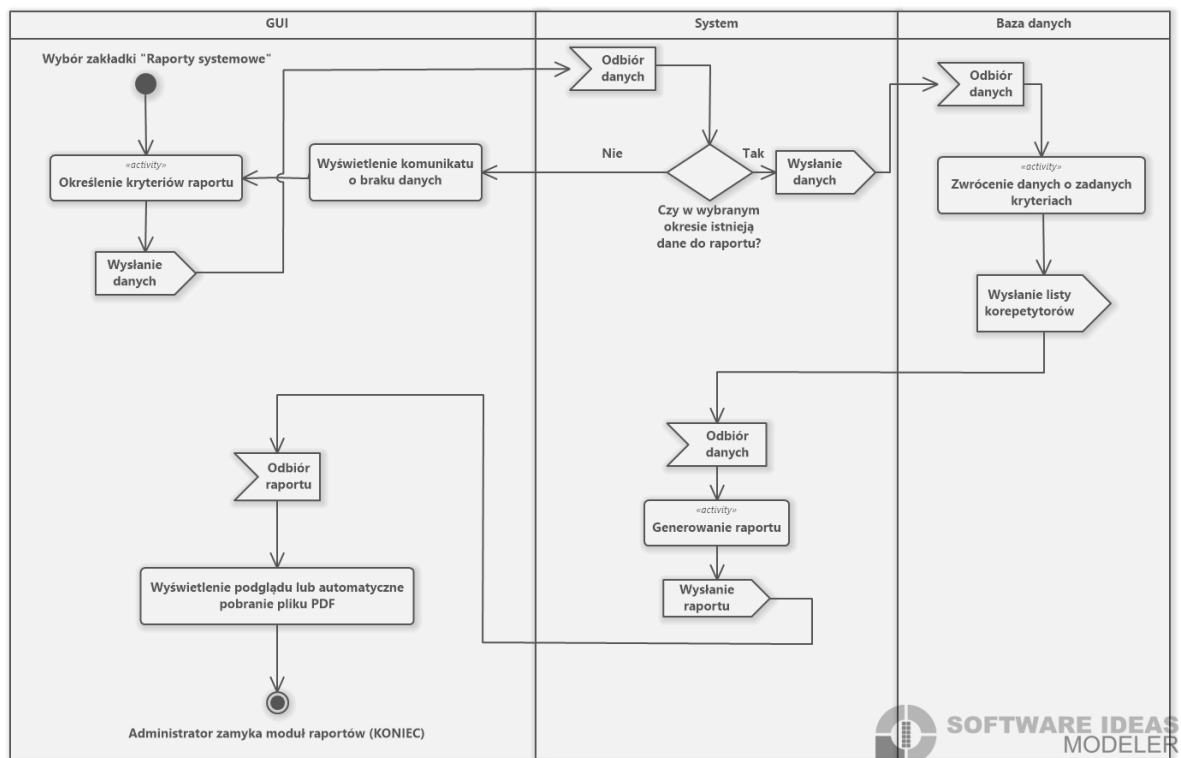


3.2. Diagramy aktywności

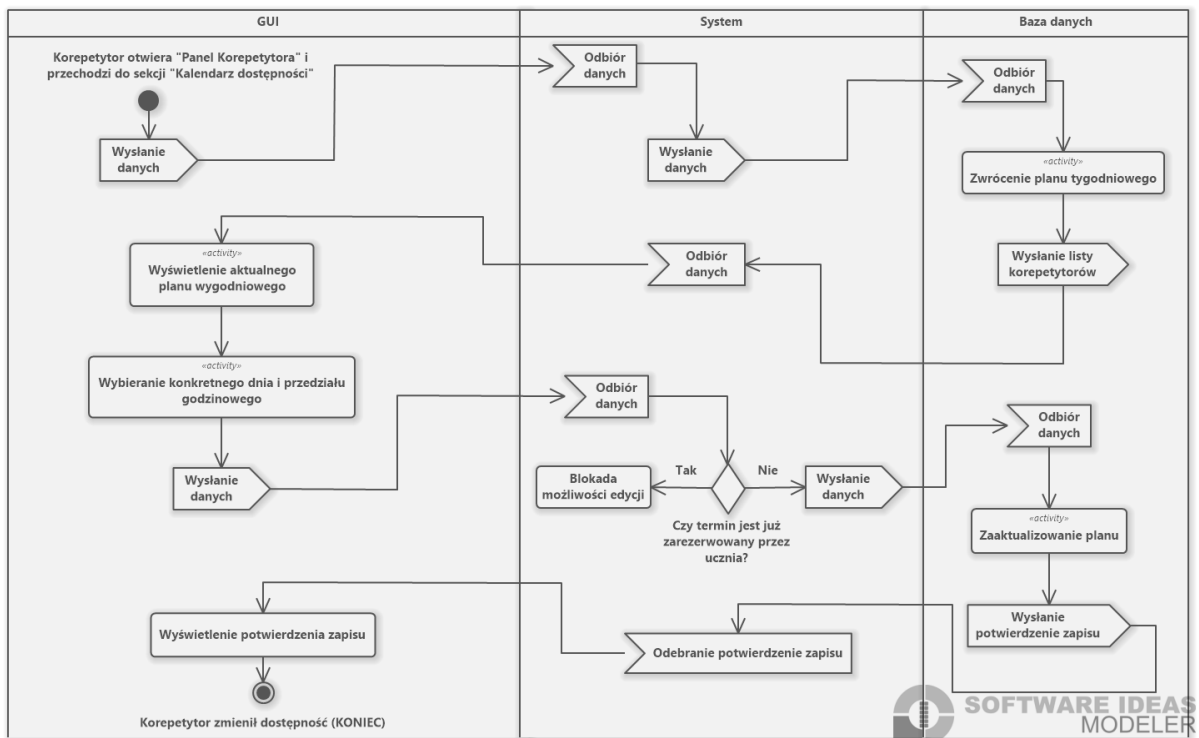
Uczeń wyszukuje korepetytorów z filtrami



Administrator generuje raport

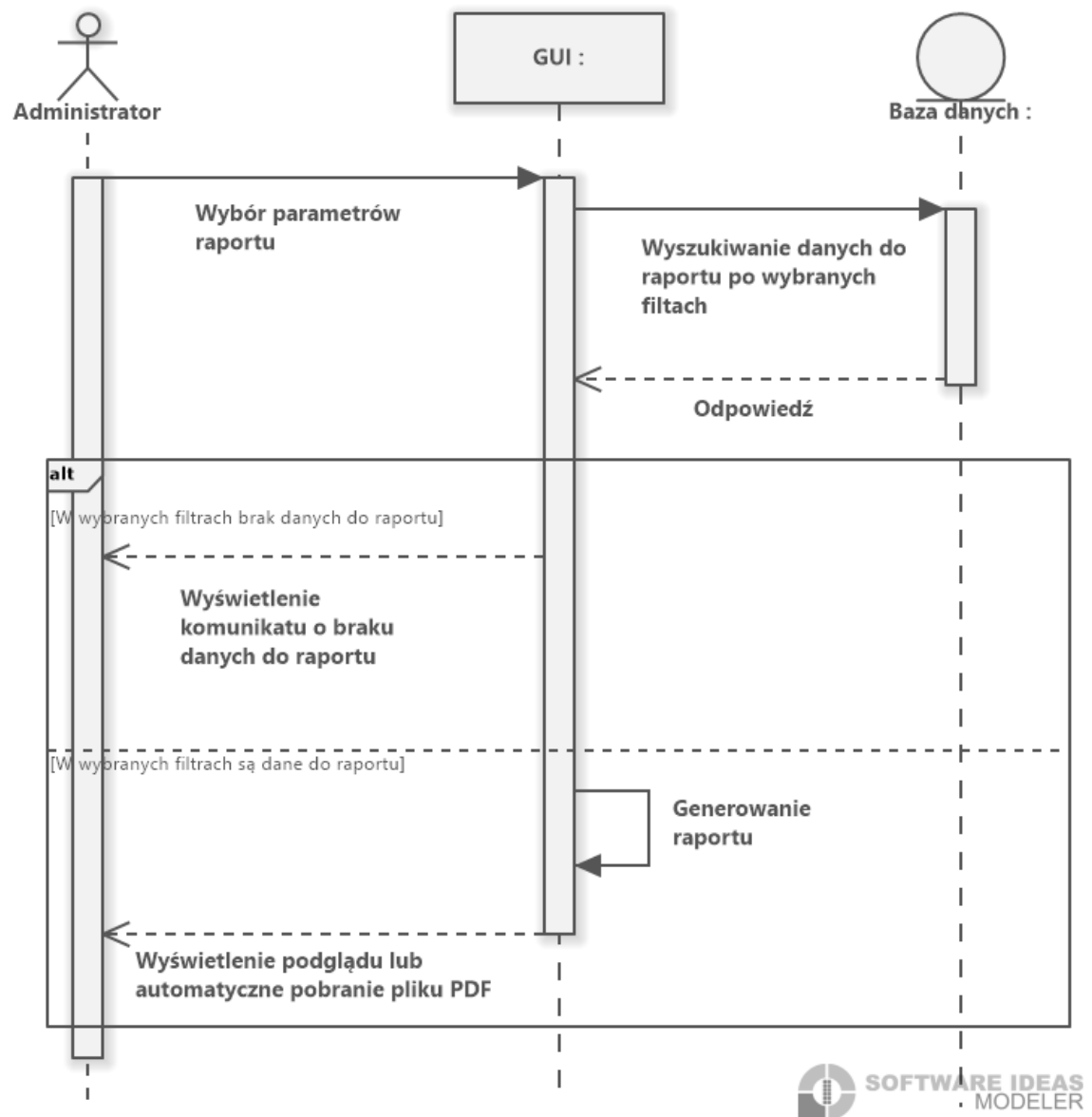


Korepetytor zarządza dostępnością

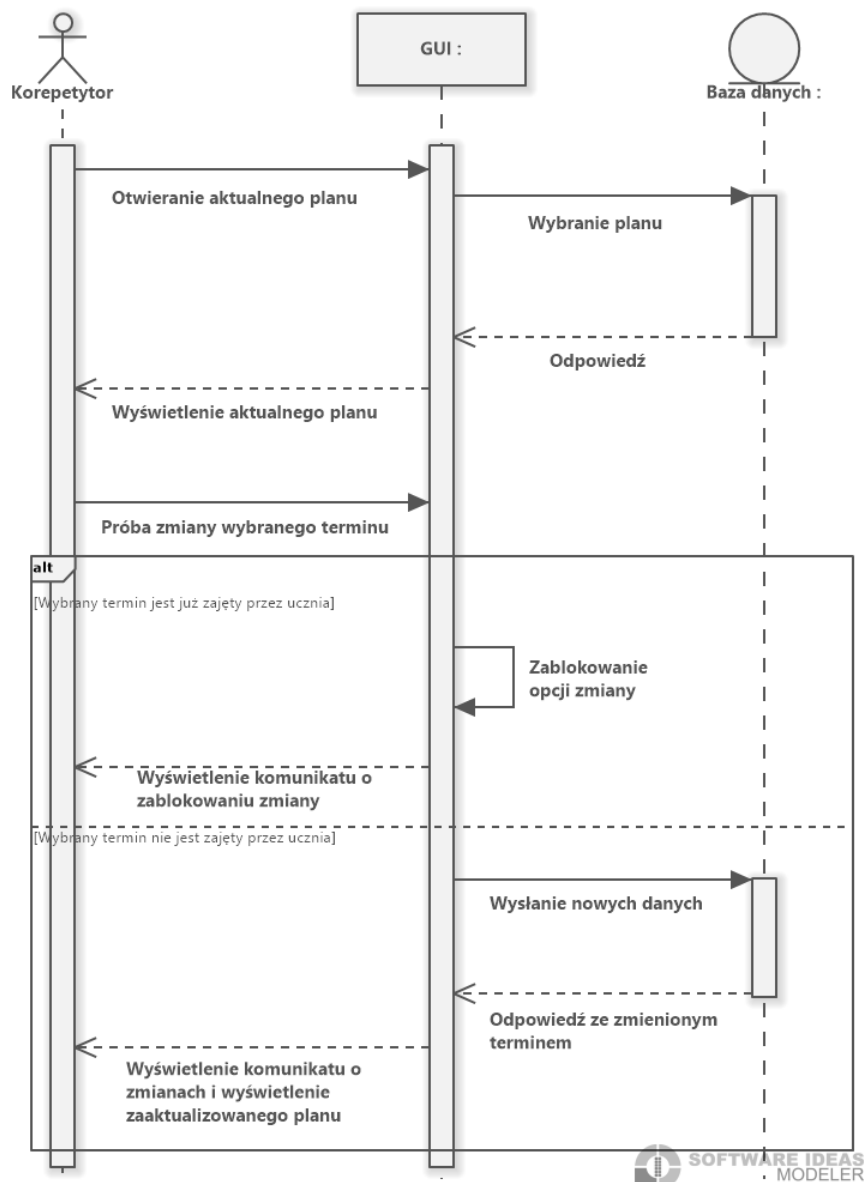


3.3. Diagramy sekwencji

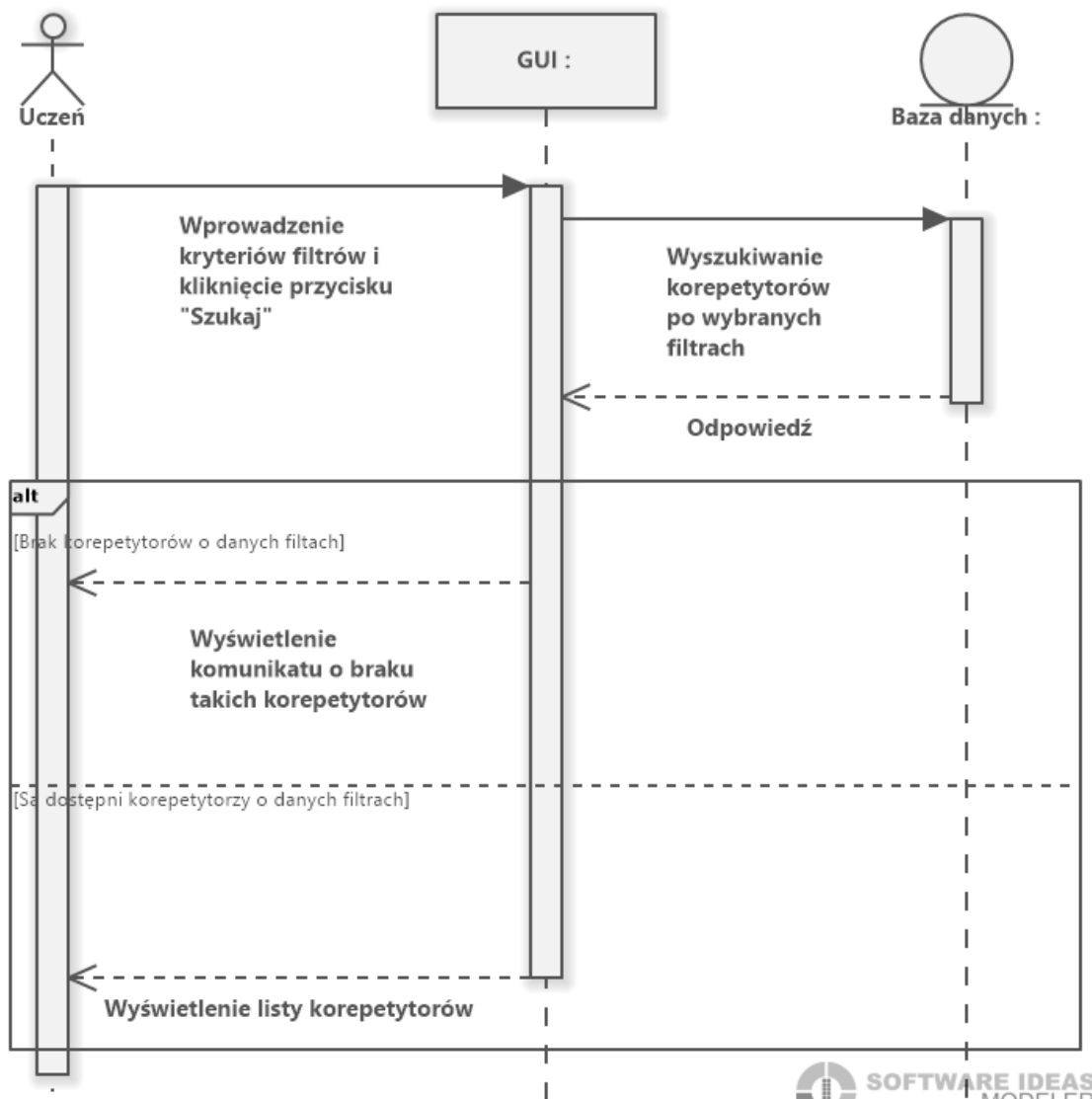
Administrator generuje raport



Korepetytor zarządza dostępnością

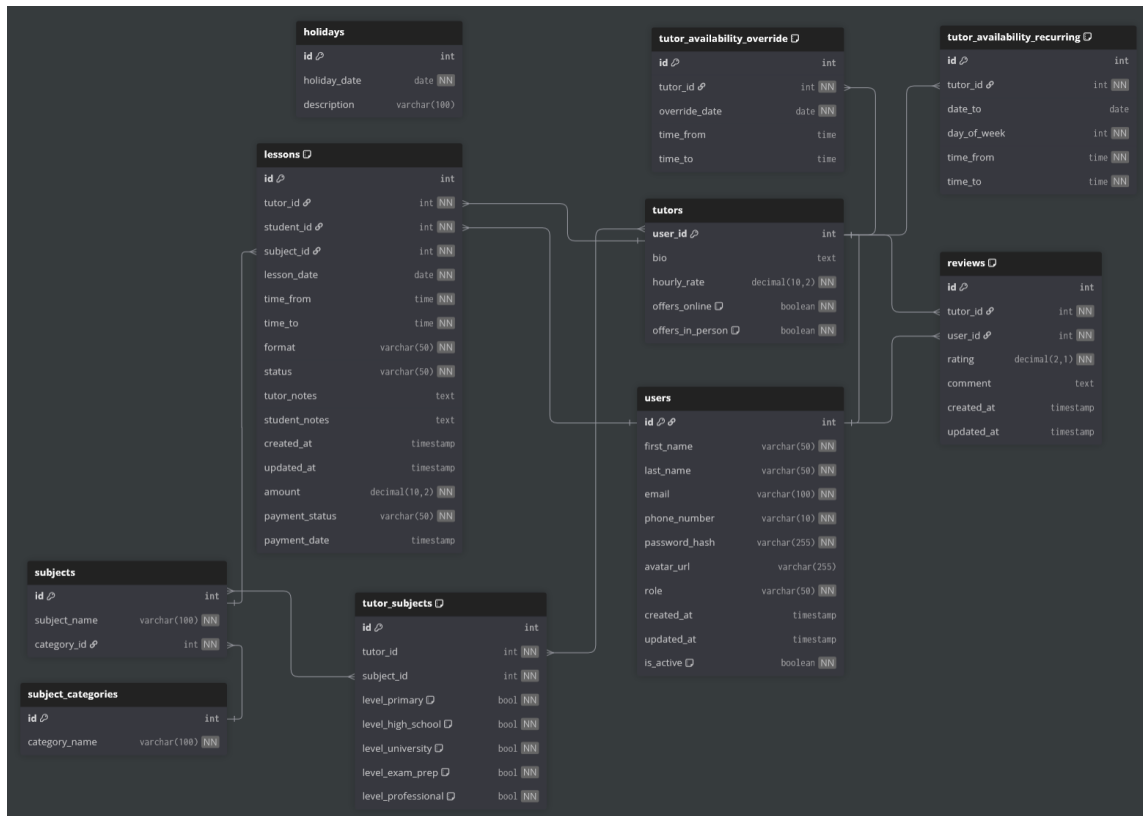


Uczeń wyszukuje korepetytorów z filtrami



4. Baza danych

4.1. Diagram ERD



4.2. Skrypt do utworzenia struktury bazy danych

```
-- Enum types

CREATE TYPE user_roles      AS ENUM ('student', 'tutor', 'admin');
CREATE TYPE lesson_formats AS ENUM ('online', 'in_person');
CREATE TYPE lesson_statuses AS ENUM ('pending', 'confirmed', 'completed', 'cancelled');
CREATE TYPE payment_statuses AS ENUM ('pending', 'paid', 'cancelled');

-- Tables

CREATE TABLE subject_categories (
    id          SERIAL          PRIMARY KEY,
    category_name VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE subjects (
    id          SERIAL          PRIMARY KEY,
    subject_name VARCHAR(100) NOT NULL UNIQUE,
    category_id INT             NOT NULL REFERENCES subject_categories(id)
);

CREATE TABLE users (
```

```

    id          SERIAL          PRIMARY KEY,
    first_name   VARCHAR(50)     NOT NULL,
    last_name    VARCHAR(50)     NOT NULL,
    email        VARCHAR(100)    NOT NULL UNIQUE,
    phone_number VARCHAR(10)     NOT NULL UNIQUE,
    password_hash VARCHAR(255)   NOT NULL,
    avatar_url   VARCHAR(255),
    user_role    user_roles      NOT NULL,
    is_active    BOOLEAN         NOT NULL DEFAULT TRUE,
    created_at   TIMESTAMP       DEFAULT NOW(),
    updated_at   TIMESTAMP       DEFAULT NOW()
);

CREATE TABLE tutors (
    user_id      INT             PRIMARY KEY REFERENCES users(id),
    bio          TEXT,
    hourly_rate  DECIMAL(10, 2) NOT NULL CHECK (hourly_rate >= 0),
    offers_online BOOLEAN        NOT NULL DEFAULT FALSE,
    offers_in_person BOOLEAN     NOT NULL DEFAULT FALSE
);

CREATE TABLE tutor_subjects (
    id          SERIAL          PRIMARY KEY,
    tutor_id    INT             NOT NULL REFERENCES tutors(user_id),
    subject_id  INT             NOT NULL REFERENCES subjects(id),
    level_primary BOOLEAN       NOT NULL DEFAULT FALSE,
    level_high_school BOOLEAN    NOT NULL DEFAULT FALSE,
    level_university BOOLEAN     NOT NULL DEFAULT FALSE,
    level_exam_prep BOOLEAN     NOT NULL DEFAULT FALSE,
    level_professional BOOLEAN   NOT NULL DEFAULT FALSE,

    UNIQUE (tutor_id, subject_id)
);

CREATE TABLE lessons (
    id          SERIAL          PRIMARY KEY,
    tutor_id    INT             NOT NULL REFERENCES tutors(user_id),
    student_id  INT             NOT NULL REFERENCES users(id),
    subject_id  INT             NOT NULL REFERENCES subjects(id),
    lesson_date DATE            NOT NULL,
    time_from   TIME            NOT NULL,
    time_to     TIME            NOT NULL,
    format      lesson_formats  NOT NULL,
    lesson_status lesson_statuses NOT NULL DEFAULT 'pending',
    tutor_notes TEXT,
    student_notes TEXT,
    amount      DECIMAL(10, 2)  NOT NULL CHECK (amount >= 0),
    payment_status payment_statuses NOT NULL DEFAULT 'pending',

```

```

payment_date    TIMESTAMP,
created_at      TIMESTAMP          DEFAULT NOW(),
updated_at      TIMESTAMP          DEFAULT NOW(),

CONSTRAINT chk_lesson_time CHECK (time_to > time_from),
CONSTRAINT chk_not_self    CHECK (tutor_id <> student_id),

UNIQUE (lesson_date, tutor_id, time_from),
UNIQUE (lesson_date, student_id, time_from)
);

CREATE TABLE reviews (
    id          SERIAL          PRIMARY KEY,
    tutor_id    INT             NOT NULL REFERENCES tutors(user_id),
    user_id     INT             NOT NULL REFERENCES users(id),
    rating      DECIMAL(2, 1)   NOT NULL CHECK (rating BETWEEN 1 AND 5),
    comment     TEXT,
    created_at  TIMESTAMP       DEFAULT NOW(),
    updated_at  TIMESTAMP       DEFAULT NOW(),

    UNIQUE (tutor_id, user_id)
);

CREATE TABLE tutor_availability_recurring (
    id          SERIAL          PRIMARY KEY,
    tutor_id    INT             NOT NULL REFERENCES tutors(user_id),
    day_of_week SMALLINT        NOT NULL CHECK (day_of_week BETWEEN 0 AND 6),
    time_from   TIME            NOT NULL,
    time_to     TIME            NOT NULL,
    date_to     DATE,           -- NULL = obowiązuje bezterminowo

    CONSTRAINT chk_recurring_time CHECK (time_to > time_from),

    UNIQUE (tutor_id, date_to, day_of_week)
);

CREATE TABLE tutor_availability_override (
    id          SERIAL          PRIMARY KEY,
    tutor_id    INT             NOT NULL REFERENCES tutors(user_id),
    override_date DATE          NOT NULL,
    time_from   TIME,           -- NULL = cały dzień niedostępny
    time_to     TIME,           -- NULL = cały dzień niedostępny

    CONSTRAINT chk_override_both_or_none
        CHECK (
            (time_from IS NULL AND time_to IS NULL) OR
            (time_from IS NOT NULL AND time_to IS NOT NULL)
        ),

```

```

CONSTRAINT chk_override_time
    CHECK (time_from IS NULL OR time_to > time_from),

    UNIQUE (tutor_id, override_date)
);

CREATE TABLE holidays (
    id          SERIAL          PRIMARY KEY,
    holiday_date DATE          NOT NULL UNIQUE,
    description VARCHAR(100)
);

-- Triggers

CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_users_updated_at
    BEFORE UPDATE ON users
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_lessons_updated_at
    BEFORE UPDATE ON lessons
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

CREATE TRIGGER trg_reviews_updated_at
    BEFORE UPDATE ON reviews
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

5. Wykorzystane technologie

- Język Kotlin 2.3.10
 - Kotlin jest głównym językiem programowania wykorzystanym do tworzenia aplikacji mobilnej na platformę Android.
- Język Java 21
 - Java jest używana po stronie backendu do implementacji serwera REST API.
- Baza danych PostgreSQL
 - PostgreSQL jest otwartym, relacyjnym systemem zarządzania bazą danych (RDBMS).
- GitHub
 - GitHub pełni rolę centralnego repozytorium kodu źródłowego z pełną historią wersjonowania (Git).
- Jira

- Jira jest narzędziem do zarządzania projektem według metodyki Scrum.
- Android SDK (Android 12+)
 - Android SDK dostarcza komponentów do obsługi nawigacji, życia aplikacji (ViewModel, LiveData/StateFlow) oraz permisji systemowych.
- SpringBoot
 - Zapewnia automatyczną konfigurację aplikacji opartej na Javie, wbudowany serwer Tomcat, obsługę REST API przez Spring MVC, integrację z Spring Security do uwierzytelniania i autoryzacji (JWT), oraz Spring Data JPA do mapowania obiektowo-relacyjnego (ORM).
- Docker
 - Docker służy do konteneryzacji aplikacji backendowej i bazy danych PostgreSQL.
- IntelliJ Idea
 - IntelliJ IDEA jest zintegrowanym środowiskiem programistycznym (IDE) stosowanym przez zespół.
- Software Ideas Modeler
 - Software Ideas Modeler to zaawansowane narzędzie do modelowania wizualnego i tworzenia diagramów UML.

6. Interfejsy systemu

6.1. Interfejs aplikacji mobilnej

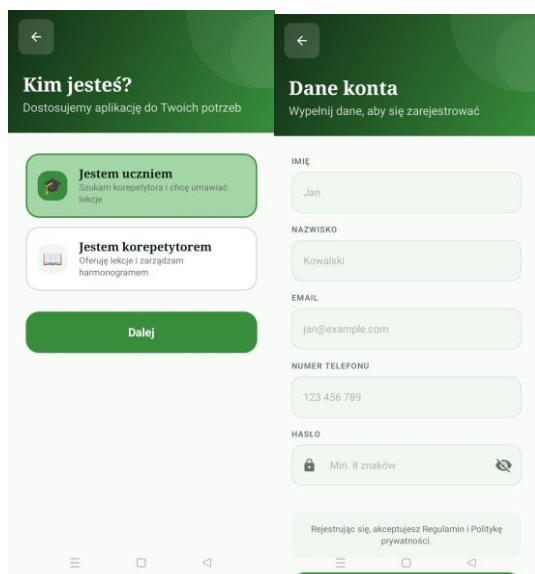
Splashscreen:

Ekran początkowy wyświetlany przy braku zapisanej sesji.

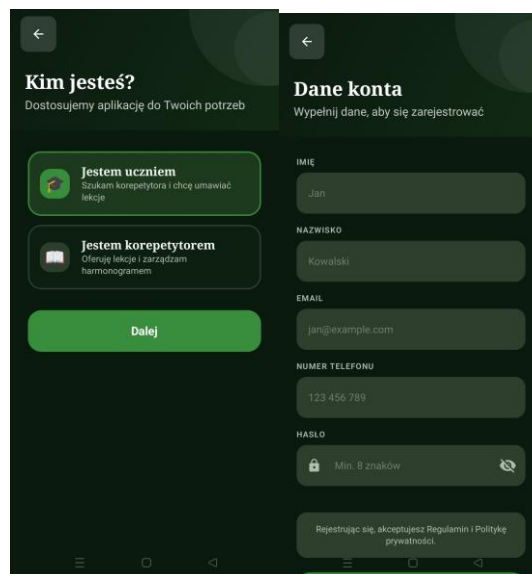


Rejestracja:

Ekran rejestracji oraz wyboru rodzaju konta.



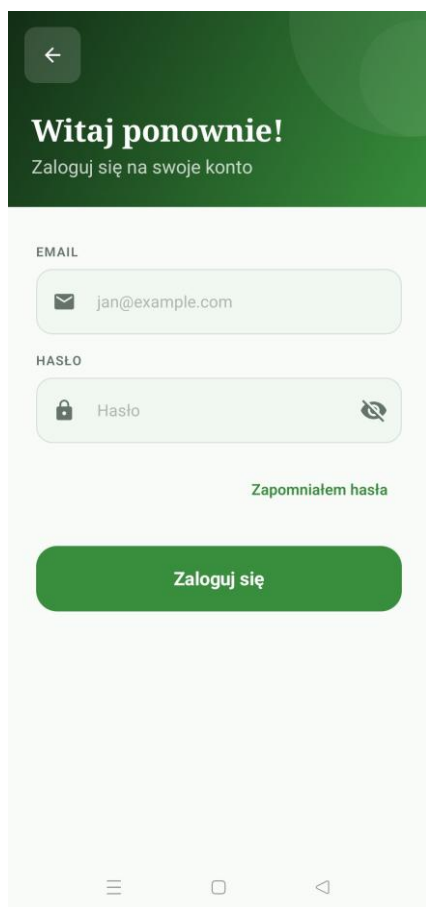
Light theme registration screen. The header has a green bar with a back arrow and the text "Kim jesteś? Dostosujemy aplikację do Twoich potrzeb". Below the header, there are two main sections. The left section, titled "Jestem uczniem" (I am a student) with a subtext "Szukam korepetytora i chcę umawiać lekcje", and "Jestem korepetytorem" (I am a tutor) with a subtext "Ofertuję lekcje i zarządzam harmonogramem", both with icons. The right section, titled "Dane konta" (Account data) with a subtext "Wypełnij dane, aby się zarejestrować", contains input fields for "IMIĘ" (Name), "NAZWISKO" (Surname), "EMAIL", "NUMER TELEFONU" (Phone number), and "HASŁO" (Password). The password field has a strength indicator "Min. 8 znaków" and a toggle for visibility. A green "Dalej" (Next) button is at the bottom. A footer bar contains a hamburger menu, a home icon, and a back arrow.



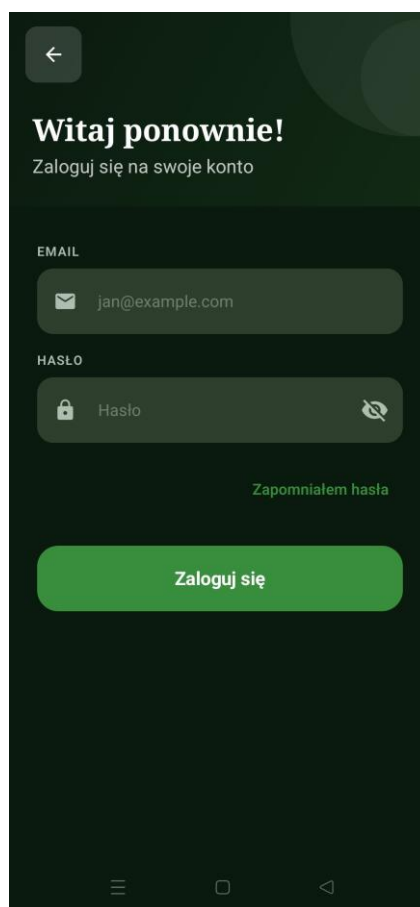
Dark theme registration screen. The layout is identical to the light theme version, but with a dark background and green accents. The header bar is dark green with white text. The input fields and buttons are dark green with white text. The footer bar is dark green with white icons.

Logowanie:

Ekran logowania.



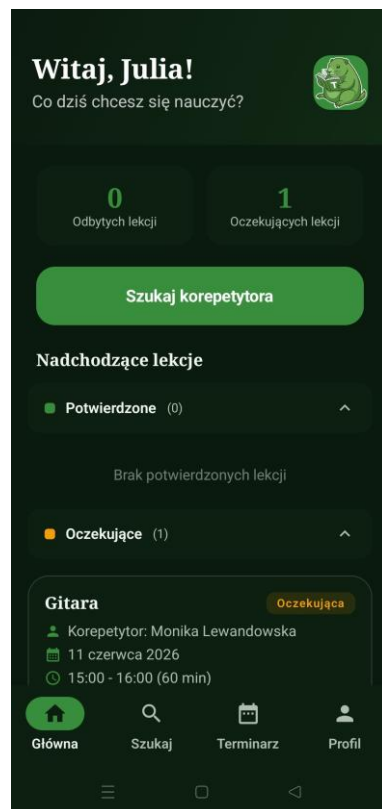
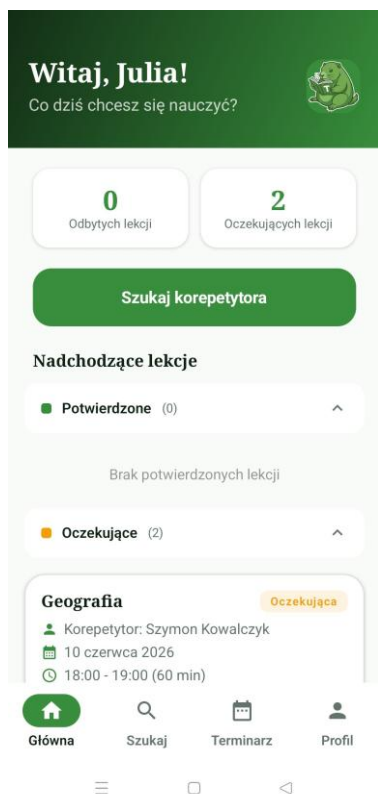
Light theme login screen. The header has a green bar with a back arrow and the text "Witaj ponownie! Zaloguj się na swoje konto". Below the header, there are two input fields: "EMAIL" and "HASŁO" (Password). The password field has a strength indicator "Min. 8 znaków" and a toggle for visibility. A green "Zaloguj się" (Login) button is at the bottom. A link "Zapomniałem hasła" (Forgot password) is located above the button. A footer bar contains a hamburger menu, a home icon, and a back arrow.



Dark theme login screen. The layout is identical to the light theme version, but with a dark background and green accents. The header bar is dark green with white text. The input fields and buttons are dark green with white text. The footer bar is dark green with white icons.

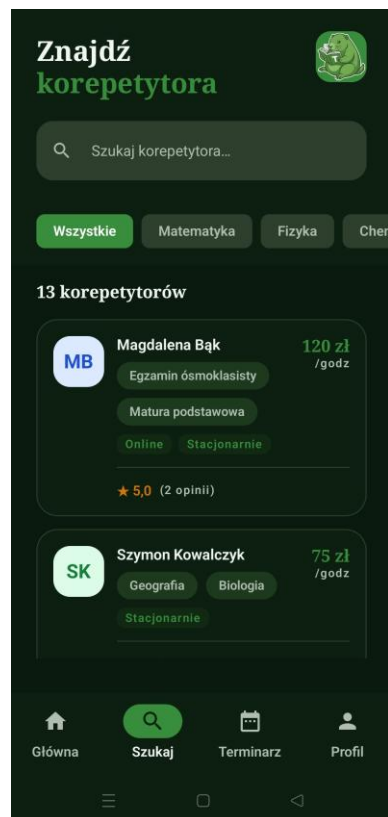
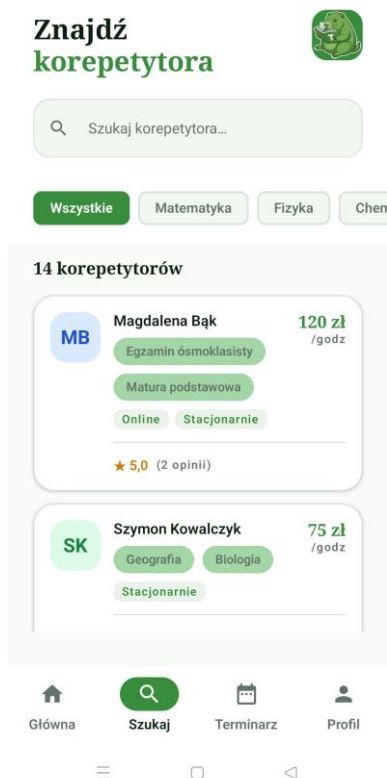
Home screen:

Ekran główny wyświetlany po zalogowaniu.



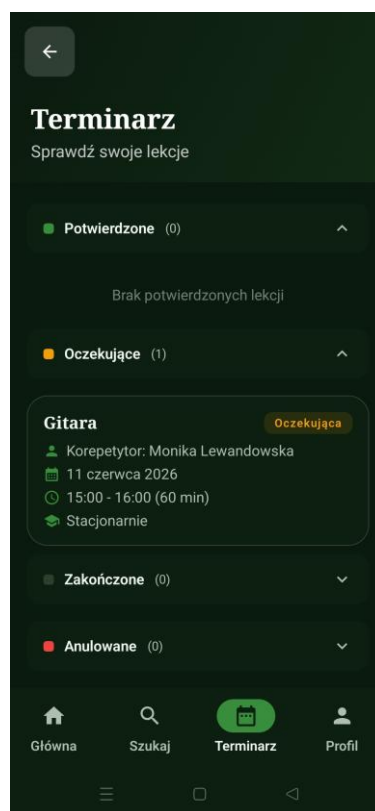
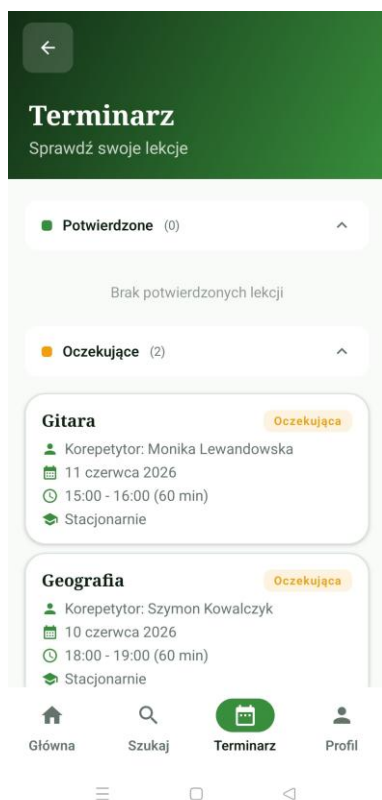
Znajdź korepetytora:

Ekran do wyszukiwania korepetytorów.



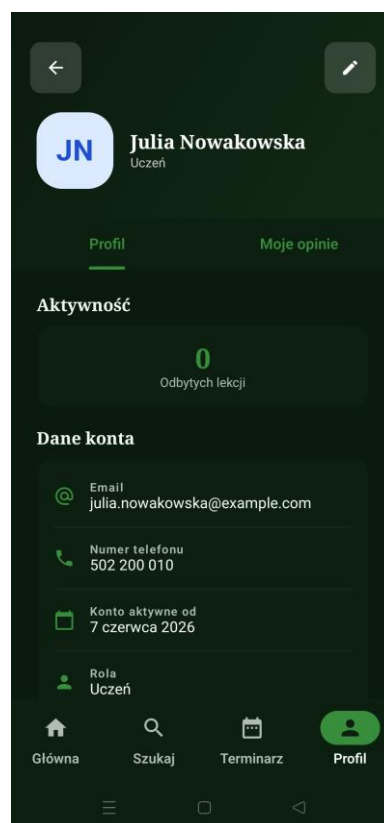
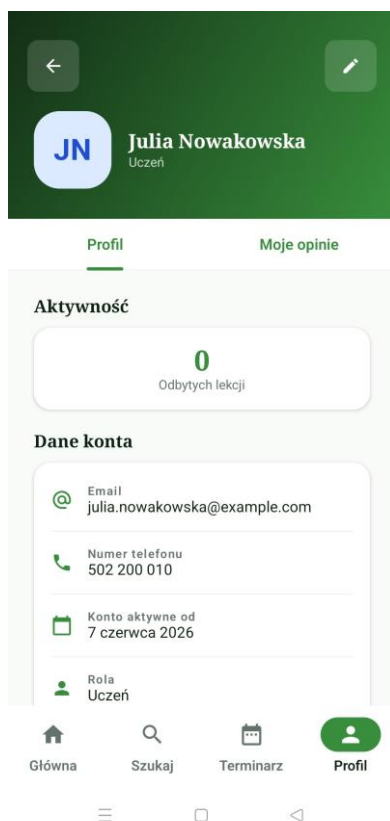
Terminarz:

Ekran z terminarzem użytkownika - zajęcia potwierdzone, oczekujące i zakończone.



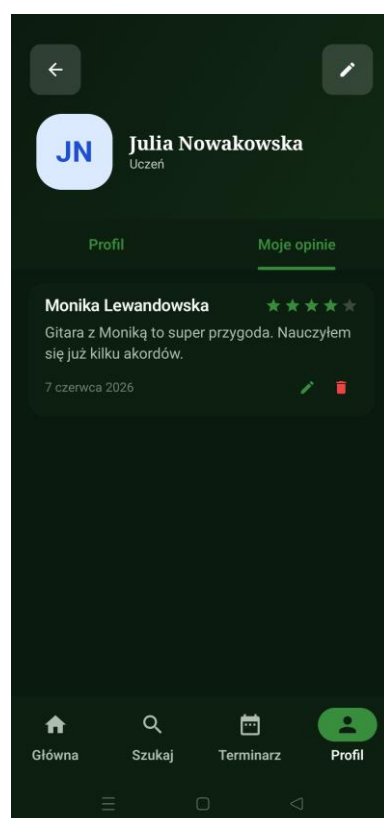
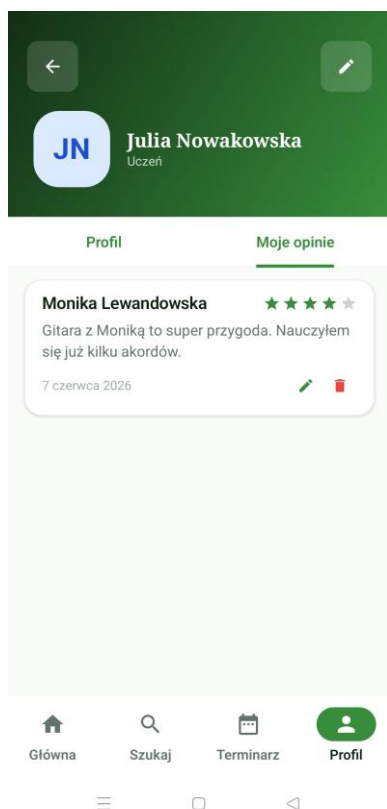
Profil:

Ekran z danymi użytkownika.



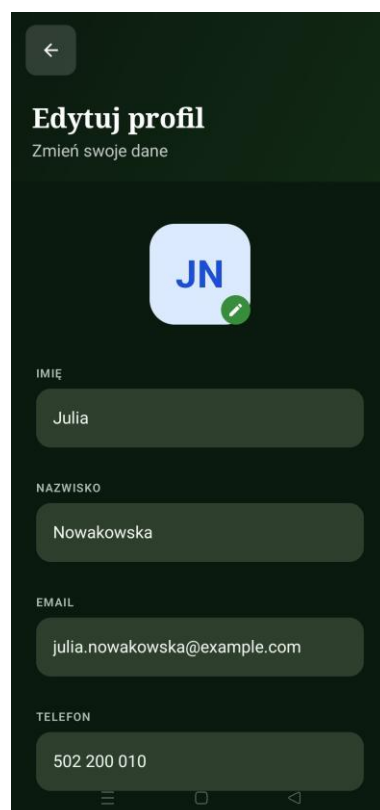
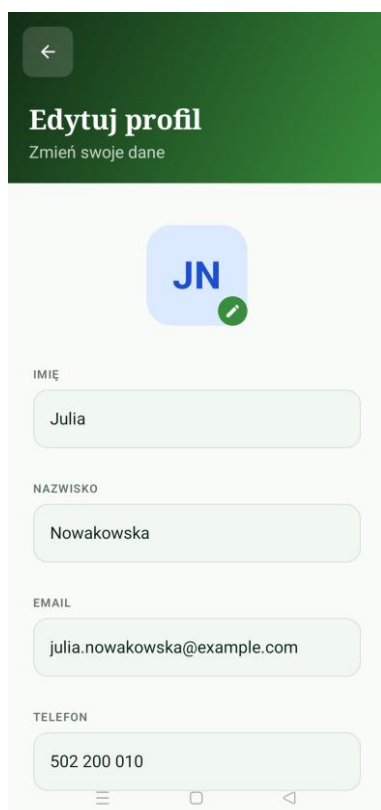
Opinie:

Lista opinii użytkownika.



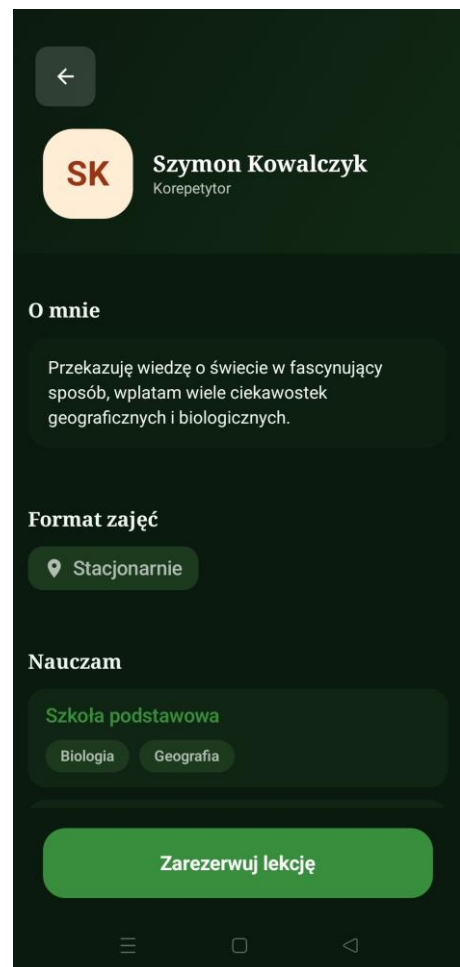
Edycja danych:

Ekran z edycją danych użytkownika.



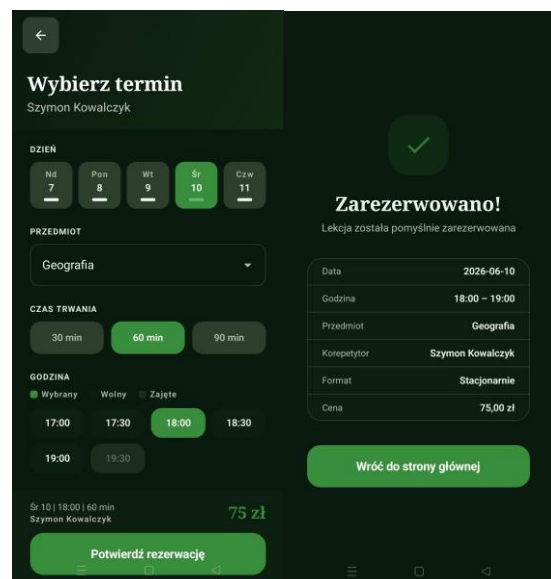
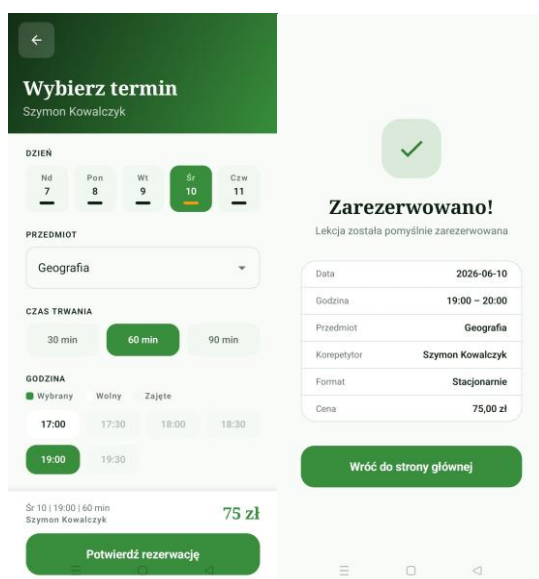
Detale korepetytora:

Ekran dokładnych informacji o korepetytorze (Profil publiczny korepetytora).



Rezerwacja lekcji

Ekran z wyborem terminu oraz potwierdzenia rezerwacji.



Harmonogram dostępności korepetytora:

Ekran do edycji harmonogramu dostępności korepetytora.

←

Harmonogram dostępności

Anna Kowalska

Tygodniowy Niedostępność

Poniedziałek + Dodaj

● 16:00 – 20:00

Wtorek + Dodaj

● 16:00 – 20:00

Środa + Dodaj

● 16:00 – 20:00

Czwartek + Dodaj

● 16:00 – 20:00

Piątek □ + Dodaj

←

Harmonogram dostępności

Anna Kowalska

Tygodniowy Niedostępność

Poniedziałek + Dodaj

● 16:00 – 20:00

Wtorek + Dodaj

● 16:00 – 20:00

Środa + Dodaj

● 16:00 – 20:00

Czwartek + Dodaj

● 16:00 – 20:00

Piątek □ + Dodaj

Edycja profilu korepetytora:

Ekran do edycji profilu korepetytora.

←

Profil korepetytora

Uzupełnij swoje dane

Informacje podstawowe

Opis (bio)

Nauczyciel matematyki z 10-letnim doświadczeniem. Specjalizuję się w przygotowaniu do matur i olimpiad.

103/2000

Stawka godzinowa (PLN)

80.0

Forma zajęć

Zajęcia online ☒

Zajęcia stacjonarnie ☒

Prowadzone przedmioty

Matematyka
Liceum, Studia, Egzaminy

Statystyka

←

Profil korepetytora

Uzupełnij swoje dane

Informacje podstawowe

Opis (bio)

Nauczyciel matematyki z 10-letnim doświadczeniem. Specjalizuję się w przygotowaniu do matur i olimpiad.

103/2000

Stawka godzinowa (PLN)

80.0

Forma zajęć

Zajęcia online ☒

Zajęcia stacjonarnie ☒

Prowadzone przedmioty

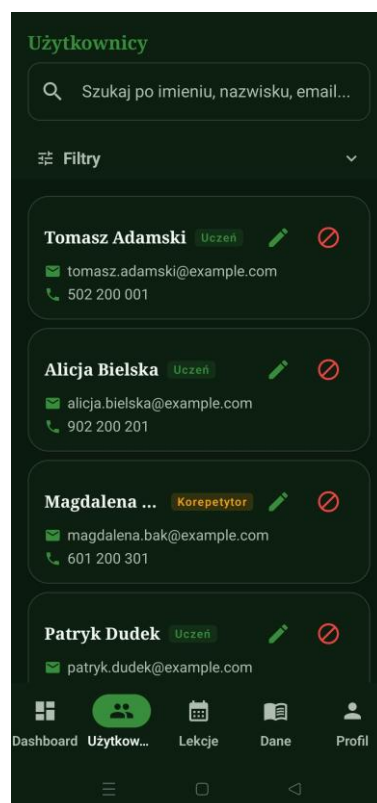
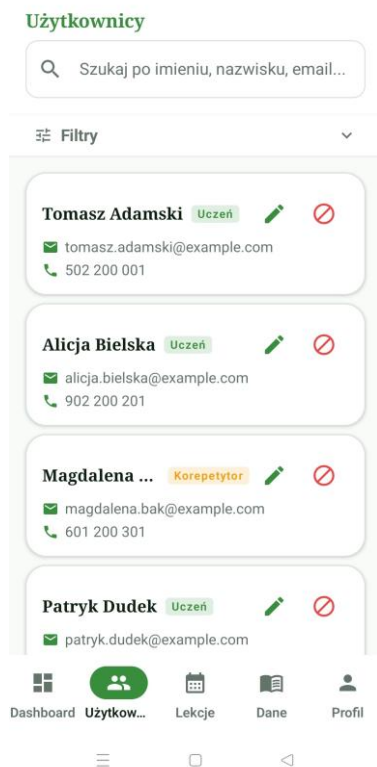
Matematyka
Liceum, Studia, Egzaminy

Statystyka

Panel administratora:
Ekran dashboard administratora.

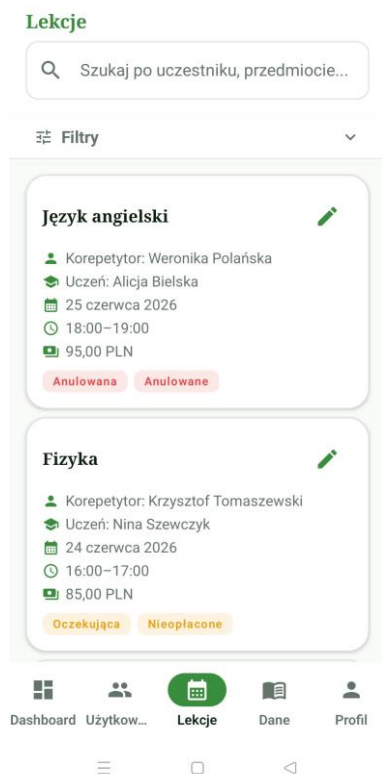


Panel użytkowników:
Ekran listy użytkowników w panelu administratora.



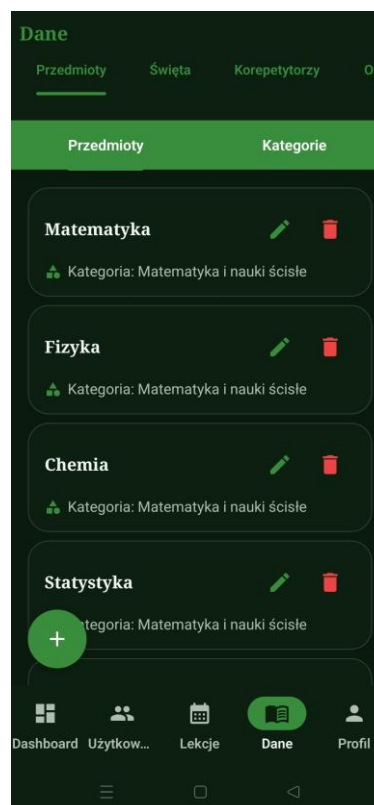
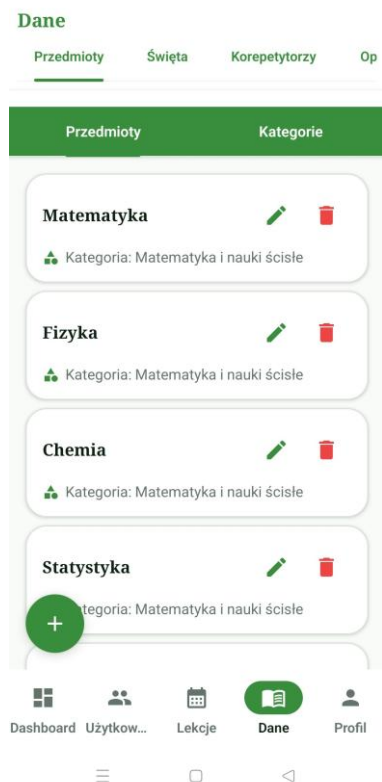
Panel lekcji:

Ekran listy lekcji w panelu administratora.



Panel danych:

Ekran listy danych w panelu administratora.



Interfejs API

endpoint	metoda	opis	dane wejściowe	dane zwracane
/api/auth/register	POST	Rejestracja nowego użytkownika	Dane rejestracyjne (np. email, hasło, imię, rola)	Dane utworzonego użytkownika
/api/auth/login	POST	Logowanie użytkownika	Email, hasło	Token JWT + dane użytkownika
/api/admin/stats	GET	Pobranie statystyk systemowych	Brak	Statystyki platformy
/api/admin/reviews?q=&rating=	GET	Pobranie opinii z filtrowaniem	Parametry q, rating	Lista opinii
/api/admin/reviews/{id}	DELETE	Usunięcie opinii	Id opinii	Status operacji
/api/users?q=&role=&active=	GET	Lista użytkowników z filtrowaniem	Parametry q, role, active	Lista użytkowników
/api/users/{id}	GET	Szczegóły użytkownika	Id użytkownika	Dane użytkownika
/api/users/{id}	PUT	Aktualizacja użytkownika	Id + dane	Zaktualizowany użytkownik

/api/users/{id}/admin	PUT	Aktualizacja użytkownika przez admina	Id + dane	Zaktualizowany użytkownik
/api/users/{id}/activate	PATCH	Aktywacja/dezaktywacja użytkownika	Id + status	Dane użytkownika
/api/users/{id}	DELETE	Usunięcie użytkownika	Id	Status operacji
/api/users/{id}/avatar	POST	Dodanie zdjęcia profilowego	Plik multipart	Dane avatara
/api/users/{id}/avatar	DELETE	Usunięcie zdjęcia profilowego	Id	Status operacji
/api/tutors/search?q=&subject=	GET	Wyszukiwanie korepetytorów	Parametry q, subject	Lista korepetytorów
/api/tutors?q=	GET	Lista korepetytorów	Parametr q	Lista korepetytorów
/api/tutors/{id}	GET	Szczegóły korepetytora	Id	Dane korepetytora
/api/tutors/{id}/subjects	GET	Przedmioty korepetytora	Id	Lista przedmiotów
/api/tutors/me	PUT	Aktualizacja własnego profilu	Dane profilu	Zaktualizowany profil
/api/tutors/me/subjects	POST	Dodanie przedmiotu	Dane przedmiotu	Dodany przedmiot
/api/tutors/me/subjects/{id}	DELETE	Usunięcie przedmiotu	Id przedmiotu	Status operacji

/api/tutors/{id}/admin	PUT	Aktualizacja korepetytora przez admina	Id + dane	Zaktualizowany profil
/api/tutors/{tutorId}/admin/subjects	POST	Dodanie przedmiotu korepetytorowi	Id + dane przedmiotu	Dodany przedmiot
/api/tutors/{tutorId}/admin/subjects/{tutorSubjectId}	DELETE	Usunięcie przedmiotu korepetytora	Id + id przedmiotu	Status operacji
/api/tutors/{tutorId}/availability/timetable	GET	Harmonogram dostępności	from, to	Harmonogram
/api/tutors/{tutorId}/availability/recurring	GET	Cykliczna dostępność	Id	Lista dostępności
/api/tutors/{tutorId}/availability/recurring	POST	Dodanie cyklicznej dostępności	Dane dostępności	Dodana dostępność
/api/tutors/{tutorId}/availability/recurring/{id}	DELETE	Usunięcie cyklicznej dostępności	Id dostępności	Status
/api/tutors/{tutorId}/availability/override	GET	Wyjątki w dostępności	Id	Lista wyjątków
/api/tutors/{tutorId}/availability/override	POST	Dodanie wyjątku	Dane wyjątku	Dodany wyjątek
/api/tutors/{tutorId}/availability/override/{id}	DELETE	Usunięcie wyjątku	Id wyjątku	Status
/api/lessons?q=&status=&paymentStatus=&format=&upcoming=	GET	Lista lekcji	Parametry filtrujące	Lista lekcji
/api/lessons/student/{studentId}	POST	Utworzenie lekcji	Dane lekcji	Utworzona lekcja
/api/lessons/student/{studentId}	GET	Lekcje ucznia	Id ucznia	Lista lekcji

/api/lessons/tutor/{tutorId}	GET	Lekcje korepetytora	Id korepetytora	Lista lekcji
/api/lessons/{lessonId}	GET	Szczegóły lekcji	Id lekcji	Dane lekcji
/api/lessons/{lessonId}/admin	PUT	Aktualizacja lekcji przez admina	Dane lekcji	Zaktualizowana lekcja
/api/lessons/{lessonId}/status	PATCH	Zmiana statusu	Nowy status	Dane lekcji
/api/lessons/{lessonId}/student-notes	PATCH	Notatki ucznia	Treść notatek	Dane lekcji
/api/lessons/{lessonId}/tutor-notes	PATCH	Notatki korepetytora	Treść notatek	Dane lekcji
/api/lessons/{lessonId}/payment	PATCH	Status płatności	Status płatności	Dane lekcji
/api/reviews/tutor/{tutorId}	GET	Opinie o korepetytorze	Id	Lista opinii
/api/reviews/student/{studentId}	GET	Opinie ucznia	Id	Lista opinii
/api/reviews/student/{studentId}	POST	Dodanie opinii	Dane opinii	Utworzona opinia
/api/reviews/{id}	PUT	Edycja opinii	Id + dane	Zaktualizowana opinia
/api/reviews/{id}	DELETE	Usunięcie opinii	Id	Status
/api/subjects	GET	Lista przedmiotów	Brak	Lista przedmiotów
/api/subjects	POST	Dodanie przedmiotu	Dane przedmiotu	Utworzony przedmiot

/api/subjects/{id}	PUT	Aktualizacja przedmiotu	Id + dane	Zaktualizowany przedmiot
/api/subjects/{id}	DELETE	Usunięcie przedmiotu	Id	Status
/api/subjects/categories	GET	Lista kategorii	Brak	Lista kategorii
/api/subjects/categories	POST	Dodanie kategorii	Dane kategorii	Utworzona kategoria
/api/subjects/categories/{id}	PUT	Aktualizacja kategorii	Id + dane	Zaktualizowana kategoria
/api/subjects/categories/{id}	DELETE	Usunięcie kategorii	Id	Status
/api/holidays	GET	Lista dni wolnych	Brak	Lista dni wolnych
/api/holidays	POST	Dodanie dnia wolnego	Dane dnia wolnego	Utworzony wpis
/api/holidays/{id}	PUT	Aktualizacja dnia wolnego	Id + dane	Zaktualizowany wpis
/api/holidays/{id}	DELETE	Usunięcie dnia wolnego	Id	Status
/api/reports/my?startDate=&endDate=&type=LESSONS&includeFields=	GET	Raport użytkownika	Parametry raportu	Raport
/uploads/**	GET	Pobieranie plików publicznych	Ścieżka pliku	Plik statyczny

7. Bezpieczeństwo i testowanie

7.1. Bezpieczeństwo

Kwestie bezpieczeństwa w aplikacji zostały rozwiązane przy wykorzystaniu frameworka Spring Security, zapewniając bezpieczne, bezstanowe zarządzanie sesjami i komunikację z klientem:

1. Haszowanie haseł:

- a. Do haszowania haseł używany jest algorytm **BCrypt**. Aplikacja definiuje BCryptPasswordEncoder jako bean w klasie konfiguracyjnej ApplicationConfig.
- b. Hasła nigdy nie są przechowywane w bazie danych jako jawny tekst. Podczas rejestracji nowego użytkownika (w AuthService), hasło przesłane przez klienta jest natychmiastowo haszowane i dopiero po tym zapisywane do encji User.

2. Autoryzacja i uwierzytelnianie (JWT - JSON Web Tokens):

- a. Projekt oparty jest na architekturze bezstanowej (zdefiniowano SessionCreationPolicy.STATELESS w klasie SecurityConfig).
- b. Zaimplementowano usługę JwtService, która odpowiada za generowanie bezpiecznych tokenów JWT, wyciąganie z nich informacji (tzw. claims) oraz weryfikację ich ważności na podstawie algorytmu podpisów HMAC SHA wykorzystującego tajny klucz.
- c. Uwierzytelnianie poszczególnych żądań HTTP następuje w niestandardowym filtrze JwtAuthFilter. Przechwytuje on każde żądanie, weryfikuje obecność tokena JWT w nagłówku i – o ile token jest prawidłowy – autoryzuje użytkownika w kontekście bezpieczeństwa Springa.

3. CORS (Cross-Origin Resource Sharing):

- a. Globalna polityka CORS została zdefiniowana w klasie SecurityConfig.
- b. Aplikacja jest skonfigurowana tak, by przyjmować zapytania z dowolnych źródeł (metoda setAllowedOriginPatterns(List.of("*"))).
- c. Dozwolony jest szeroki wachlarz metod HTTP (GET, POST, PUT, DELETE, PATCH, OPTIONS) oraz wszystkie możliwe nagłówki, co ułatwia współpracę z aplikacją frontendową, uwzględniając przy tym wsparcie dla uwierzytelniania poprzez ciasteczka/nagłówki (włączone setAllowCredentials(true)).

7.2. Testy

- Testy jednostkowe (Unit Tests): Skupiają się one na warstwie serwisów (np. AuthServiceTest, UserServiceTest, LessonServiceTest). Zależności do bazy danych i innych modułów są w nich izolowane za pomocą „zaślepek” (mocków), co pozwala przetestować czystą logikę biznesową.
- Testy integracyjne / kontrolerów (WebMvc): Sprawdzają działanie poszczególnych punktów końcowych (endpoints) REST API bez podnoszenia pełnego środowiska serwerowego.

Odpowiadają za nie pliki takie jak `AuthControllerTest`, `UserControllerTest` itd., pozwalając sprawdzić np. mapowanie JSON-ów, poprawne ścieżki routingu i kody odpowiedzi HTTP.

- Testy manualne: Do doraźnego wywoływania i testowania odpowiedzi z API wykorzystywany jest standardowo Postman.
- Narzędzia wykorzystane do testów w kodzie:
- JUnit 5: Główny framework testowy służący do definiowania cyklu życia testów i uruchamiania przypadków testowych.
- Mockito: Narzędzie do tworzenia mocków (fałszywych obiektów), używane masowo w testach serwisów (np. `@Mock`, `@InjectMocks`).
- MockMvc: Komponent Springa wykorzystywany do symulowania żądań HTTP (metody typu `post(...)`, `get(...)`) oraz weryfikowania statusów zwracanych w odpowiedziach z kontrolerów REST.
- AssertJ: Dostarcza przyjazne asercje (np. fluent API w metodach takich jak `assertThat()`, `assertThatThrownBy()`), co zwiększa czytelność tego, co dokładnie sprawdzają testy.
- Przykłady konkretnych przypadków testowych (na bazie pakietu `auth`):
- Logowanie:
- Testowanie poprawnego scenariusza uwierzytelniania: test weryfikuje czy podanie prawidłowego e-maila i hasła skutkuje wygenerowaniem tokena JWT i zwróceniem obiektu typu `AuthResponse`. Testowany jest również przypadek weryfikujący poprawność kodu HTTP 200 OK z endpointu `/api/auth/login`.
- Rejestracja:
- Test sukcesu, który sprawdza przypisanie odpowiedniej roli (np. `STUDENT`) z powrotem do DTO, zwrócenie ważnego tokena oraz poprawne wywołanie metody zapisu w repozytorium użytkowników.
- Haszowanie podczas rejestracji: Projekt posiada dedykowany test (`register_passwordIsHashed`), upewniający się, że zachowywane hasło do bazy danych przed zapisem bezwzględnie zostaje przepuszczone przez `BCryptPasswordEncoder`.
- Błędne dane (walidacja biznesowa):
- Testowanie konfliktów danych, na przykład rejestracja konta na zajęty już adres e-mail (`register_emailAlreadyTaken_throwsBusinessValidationException`) lub na zajęty numer telefonu. Testy te weryfikują czy backend nie wywala błędu krytycznego, lecz kulturalnie wyrzuca niestandardowy wyjątek `BusinessValidationException` z odpowiednim komunikatem dla klienta.

8. Załączniki

Manual.docx